A REPORT ON NETWORK TOPOLOGY SIMULATOR

SUBMITTED TO THE SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE, IN THE PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE AWARD OF THE DEGREE OF

THIRD YEAR COMPUTER ENGINEERING

BY

TIRTHA ZOLDEO
MAYUR UMMAKANT ZOPE
YASH KISHOR CHAUDHARY
PREM NARENDRA BORDE

UNDER THE GUIDANCE OF

PROF. PRITAM AHIRE

Nutan Maharashtra Vidya Prasarak Mandal's
NUTAN MAHARASHTRA INSTITUTE OF ENGINEERING & TECHNOLOGY, PUNE

Samarth Vidya Sankul, Vishnupuri,
Talegaon Dabhade, Maharashtra 410507

ACADEMIC YEAR : 2025-2026

NOV-DEC 2025

CERTIFICATE

This is to certify that the mini project titled

NETWORK TOPOLOGY SIMULATOR

Submitted by

PREM NARENDRA BORDE
TIRTHA ZOLDEO
MAYUR UMAKANT ZOPE
YASH KISHOR CHAUDHARY

under my guidance in partial fulfillment of the requirements for the award of the degree of Third Year Computer Engineering at Nutan Maharashtra Institute of Engineering and Technology, Talegaon Dabhade.

Prof. Pritam Ahire	**Dr. Rohini Hanchate**	**Dr. Pramod Patil**
Subject Incharge	**Head of Computer Engineering**	**Principal**

Date:

Place: Talegaon, Pune

1. PROJECT OVERVIEW

1.1 Title

Network Topology Simulator – Interactive Network Visualization and Packet Transmission Platform

1.2 Abstract

This project introduces an interactive simulation environment for visualizing and understanding different network topologies and their data transmission behaviours. The system provides an intuitive interface for configuring topologies, selecting nodes, and visualizing packet routing in real time. It simulates Star, Bus, Ring, and Mesh architectures, each adhering to unique communication patterns. Designed primarily for students, educators, and network enthusiasts, the simulator bridges the gap between theoretical network design and practical visualization. Built using React + TypeScript + Vite, it combines modern front-end frameworks, real-time SVG rendering, and dynamic animation logic to create an engaging learning experience.

1.3 Objectives

- Demonstrate core network architectures through real-time visualization.
- Enable users to interactively simulate data flow between nodes.
- Highlight topology-specific routing characteristics.
- Provide detailed simulation logs and real-time transmission feedback.
- Offer a responsive and aesthetic user interface.
- Serve as an educational tool for understanding data communication systems.

1.4 Motivation

- Learning computer networks is often abstract and theoretical. Traditional classroom diagrams fail to illustrate dynamic concepts like packet traversal, collisions, and routing paths. Hence, this simulator was designed to:
- Make network learning interactive.
- Bridge theoretical knowledge with visualization.
- Encourage experimentation and observation through real-time simulation.

2. TECHNOLOGY STACK

2.1 Frontend Environment

Component	Technology	Description
Framework	React (v18.3.1)	Component-based, dynamic UI creation
Language	TypeScript	Strongly typed JavaScript for safer code
Build tool	Vite	Fast bundler with hot module replacement
Styling	Tailwind CSS	Utility first CSS framework for rapid design
UI Library	Shadcn/UI + Radix UI	Prebuild, accessible components
State Management	React Hooks	For managing dynamic UI state
Routing	React Router DOM	Handles page navigation within SPA

2.2 Supplementary Libraries

- **Lucide React:** Modern icons for UI buttons and controls
- **Zod:** Schema-based form validation
- **React Hook Form:** Manages input forms efficiently
- **Sonner:** For displaying toasts and notifications
- **Class Variance Authority (CVA):** Simplifies conditional styling.

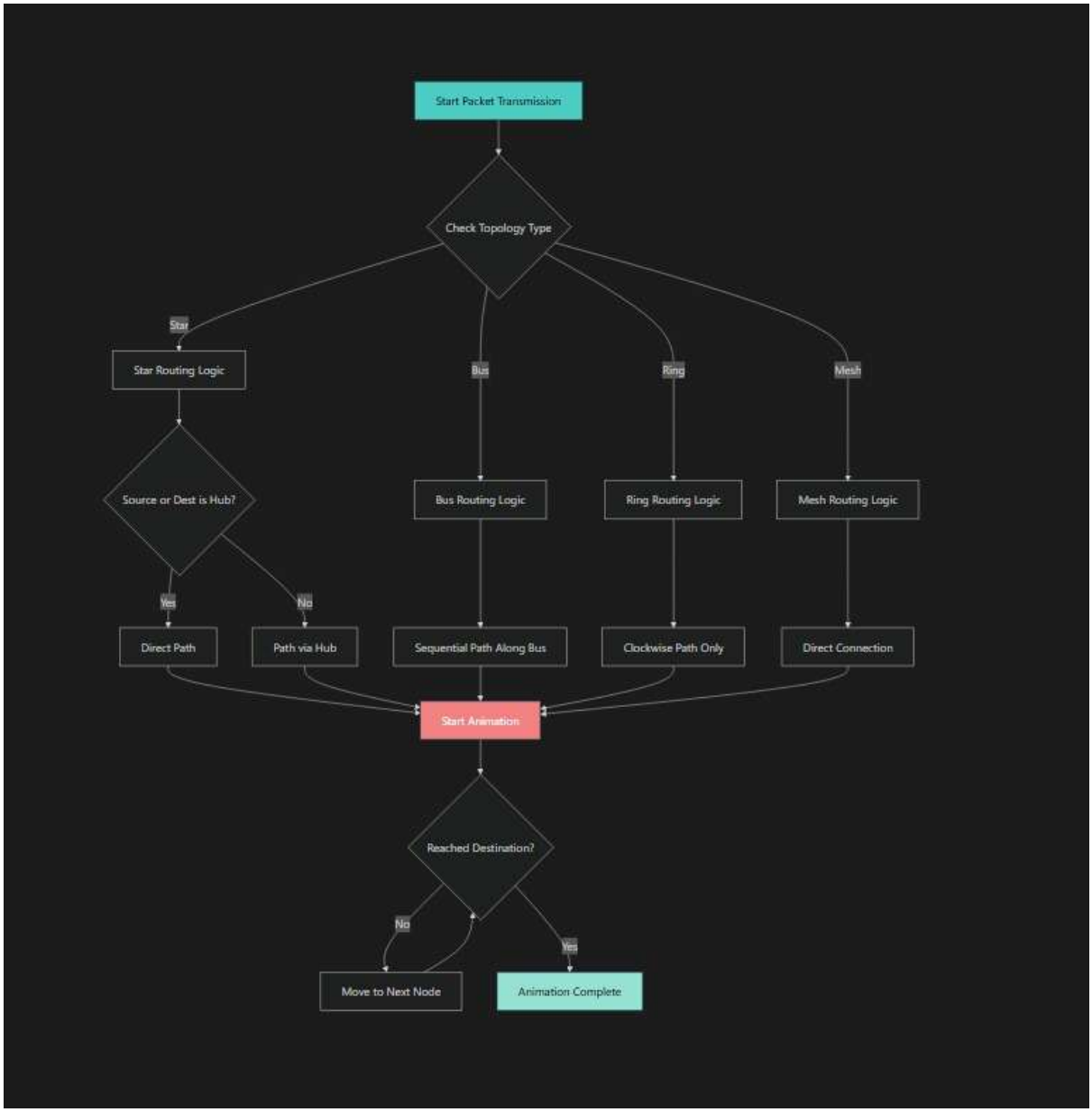
2.3 Tools & Environment

- *Code Editor: Visual Studio Code*
- *Version Control: Git + GitHub*
- *Testing Environment:* Browser (Chrome/Edge)

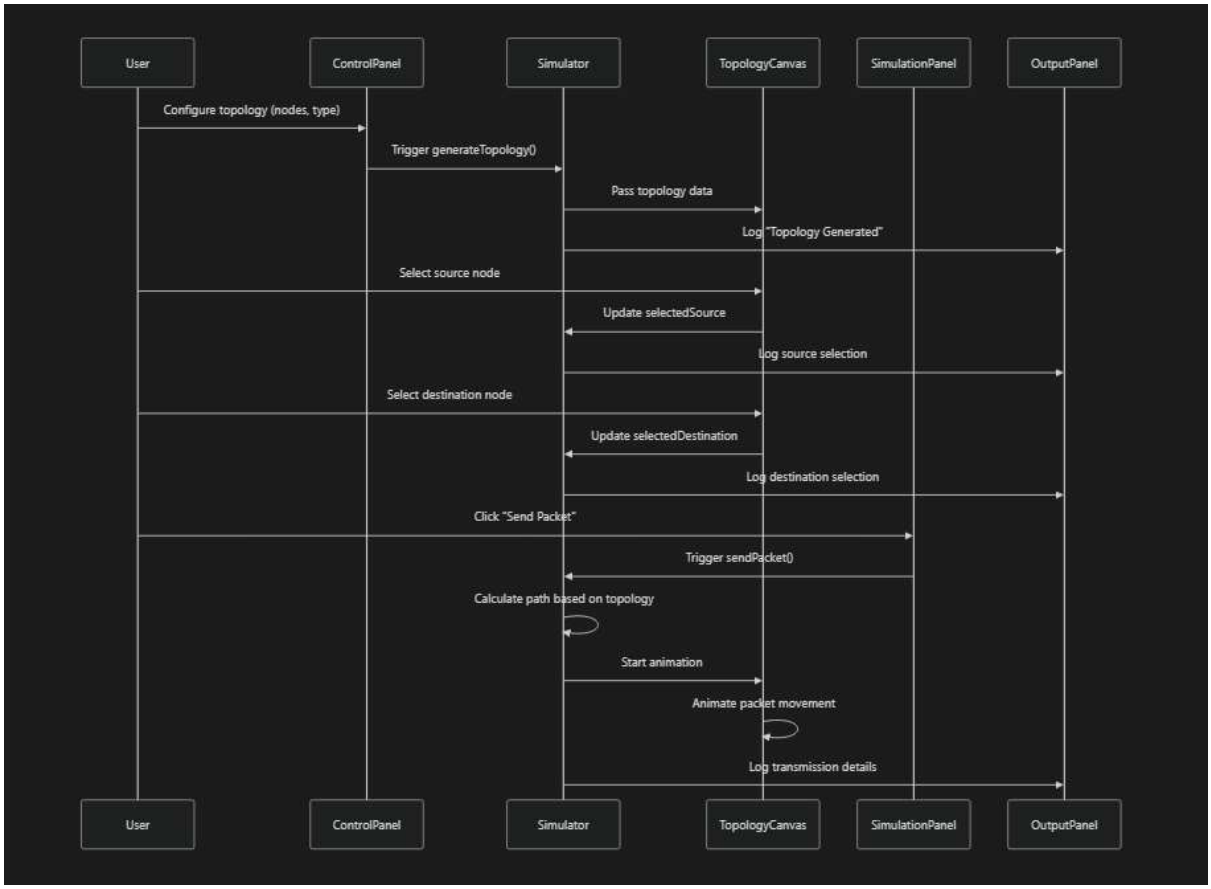
- *Deployment:* Vercel or Netlify.

3. SYSTEM ARCHITECTURE

3.1 Component Structure



3.2 Data Flow


User
ControlPanel
Simulator
TopologyCanvas
SimulationPanel
OutputPanel
Configure topology (nodes, type)
Trigger generateTopology()
Pass topology data
Log "Topology Generated"
Select source node
Update selectedSource
Log source selection
Select destination node
Update selectedDestination
Log destination selection
Click "Send Packet"
Trigger sendPacket()
Calculate path based on topology
Start animation
Animate packet movement
Log transmission details
User
ControlPanel
Simulator
TopologyCanvas
SimulationPanel
OutputPanel

4. FEATURES & FUNCTIONALITY

4.1 Network Topology Generation

4.1.1 Star Topology

- *Structure*: Central hub connected to all peripheral nodes
- *Characteristics*:
  - One central hub node
  - 2-10 peripheral nodes arranged in a circular pattern
  - Direct connections from hub to each node
- *Use Case*: Centralized network architecture (e.g., home networks, small offices)

4.1.2 Bus Topology

- *Structure*: Linear arrangement of nodes along a single bus line
- *Characteristics*:
  - Sequential node placement
  - Each node connected to adjacent nodes
  - Data travels along the bus line
- *Use Case*: Simple network setups, industrial automation

4.1.3 Ring Topology

- *Structure*: Circular arrangement with unidirectional data flow
- *Characteristics*:
  - Nodes arranged in a circle
  - **Unidirectional**: Data always travels clockwise
  - Each node connected to next node in sequence
- *Use Case*: Token ring networks, metropolitan area networks

4.1.4 Mesh Topology

- *Structure*: Fully interconnected network
- *Characteristics*:
  - Every node directly connected to every other node
  - Maximum redundancy and reliability
  - Grid-based visual arrangement
- *Use Case*: High-availability systems, backbone networks

4.2 Packet Simulation & Animation

4.2.1 Path Calculation Algorithm

```
flowchart TD

    Start[Start Packet Transmission] --> CheckTopology{Check Topology Type}
```

```
CheckTopology -->|Star| StarLogic[Star Routing Logic]
CheckTopology -->|Bus| BusLogic[Bus Routing Logic]
CheckTopology -->|Ring| RingLogic[Ring Routing Logic]
CheckTopology -->|Mesh| MeshLogic[Mesh Routing Logic]

StarLogic --> StarCheck{Source or Dest is Hub?}
StarCheck -->|Yes| DirectPath[Direct Path]
StarCheck -->|No| ViaHub[Path via Hub]

BusLogic --> Sequential[Sequential Path Along Bus]

RingLogic --> Clockwise[Clockwise Path Only]

MeshLogic --> Direct[Direct Connection]

DirectPath --> Animate[Start Animation]
ViaHub --> Animate
Sequential --> Animate
Clockwise --> Animate
Direct --> Animate

Animate --> Moving{Reached Destination?}
Moving -->|No| Continue[Move to Next Node]
Continue --> Moving
Moving -->|Yes| Complete[Animation Complete]
```

4.2.2 Animation Features

- *Real-time Packet Movement*: Green glowing packet (radius) moves at 3 pixels per frame (≈60fps)
- *Path Highlighting*: Active connections glow and animate with dashed lines
- *Smooth Interpolation*: Calculated movement vectors ensure smooth transitions
- *Completion Detection*: Animation continues until packet physically reaches destination coordinates

4.3 User Interface Components

4.3.1 Control Panel

- Node count input (2−10 nodes)
- Topology type selector with visual icons
- Generate button with cyberpunk styling
- Topology information guide

4.3.2 Topology Canvas

- **SVG-based rendering** for precise graphics
- Grid pattern background for depth
- Interactive node selection
- Color-coded node states:
    - *Green*: Source node
    - *Blue*: Destination node
    - *Cyan*: Available nodes
- Real-time topology statistics display

4.3.3 Simulation Panel

- Source/destination node displays
- Send packet button with loading states
- Quick guide for user interaction
- Disabled during active animations

4.3.4 Output Panel

- Scrollable log display (50 entry limit)
- Timestamped entries
- Color-coded log types:
    - ✓ Success (green)
    - △ Warning (yellow)
    - × Error (red)
    - i Info (cyan)
- Clear logs functionality

5. METHODOLOGY

5.1 Development Approach

Phase 1: Design & Planning

1. Research network topology characteristics
2. Define component architecture
3. Design UI/UX with cyberpunk theme
4. Plan state management strategy

Phase 2: Core Implementation

1. Set up React + TypeScript + Vite environment
2. Implement topology generation algorithms
3. Create base UI components using shadcn/ui
4. Develop SVG-based canvas rendering

Phase 3: Packet Simulation

1. Design path calculation algorithms for each topology
2. Implement animation state management
3. Create smooth packet movement system
4. Add path highlighting functionality

Phase 4: Polish & Refinement

1. Add logging system
2. Implement topology-specific routing rules
3. Ensure unidirectional ring topology compliance
4. Optimize animation performance
5. Add user feedback mechanisms

5.2 Algorithm Design

5.2.1 Star Topology Routing

```
if (source === HUB || destination === HUB) {

    path = [source → destination] // Direct

} else {

    path = [source → HUB → destination] // Via hub

}
```

5.2.2 Bus Topology Routing

start = \min(\text{source}, \text{destination})

end = \max(\text{source}, \text{destination})

path = [\text{start}, \text{start}+1, \dots, \text{end}]

if (\text{source} > \text{destination}) \text{reverse}(\text{path})

5.2.3 Ring Topology Routing (Unidirectional)

```
path = []

current = source

while (current !== destination) {

    path.push(current)

    current = (\text{current} + 1) \pmod{\text{totalNodes}} // Clockwise only

}

path.push(destination)
```

5.2.4 Mesh Topology Routing

path = [source → destination] // Direct connection

5.3 Animation Implementation

The packet animation uses a frame-based approach:

1. *Initialization*: Calculate complete path based on topology rules
2. *Movement Loop*: 60fps interval (16ms) updates packet position
3. *Vector Calculation*: Compute direction and distance to next node
4. *Interpolation*: Move packet incrementally (3px/frame) toward target
5. *Node Transition*: Snap to node when within threshold distance
6. *Completion*: Verify arrival at final destination before stopping

6. WORKING DEMONSTRATION

6.1 User Workflow

```
journey
  title User Journey - Packet Simulation
  section Setup
    Configure node count: 5: User
    Select topology type: 5: User
    Generate topology: 5: User, System
  section Node Selection
    Click source node: 4: User
    View source highlighted: 5: System
    Click destination node: 4: User
    View destination highlighted: 5: System
  section Simulation
    Click Send Packet: 5: User
    Calculate path: 5: System
    Animate packet: 5: System
    View logs: 4: User
  section Analysis
    Review transmission path: 4: User
    Check hop count: 4: User
    Analyze routing: 5: User
```

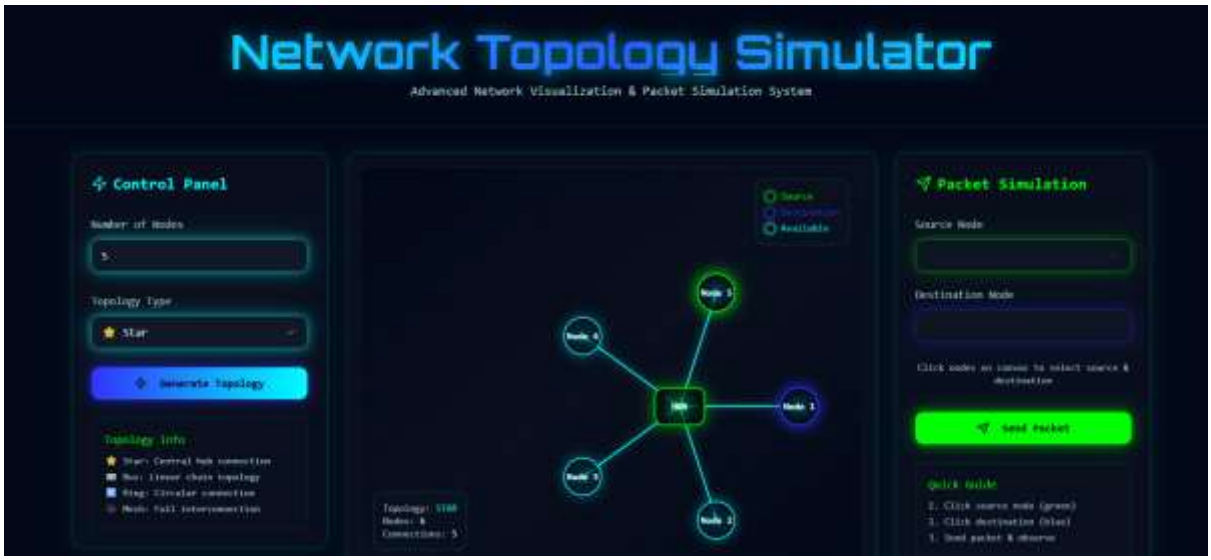
6.2 Example Simulation Scenarios

Scenario 1: Star Topology (5 nodes)

- Configuration: 1 Hub +5 peripheral nodes
- Source: Node 2→ Destination: Node 4
- Path: Node 2→HUB→Node 4
- Hops: 2
- Time: ≈2 seconds

Scenario 2: Ring Topology (6 nodes, Unidirectional)

- Configuration: 6 nodes in circle
- Source: Node 1→ Destination: Node 5
- Path: Node 1→Node 2→Node 3→Node 4→Node 5
- Hops: 4
- Direction: Clockwise only

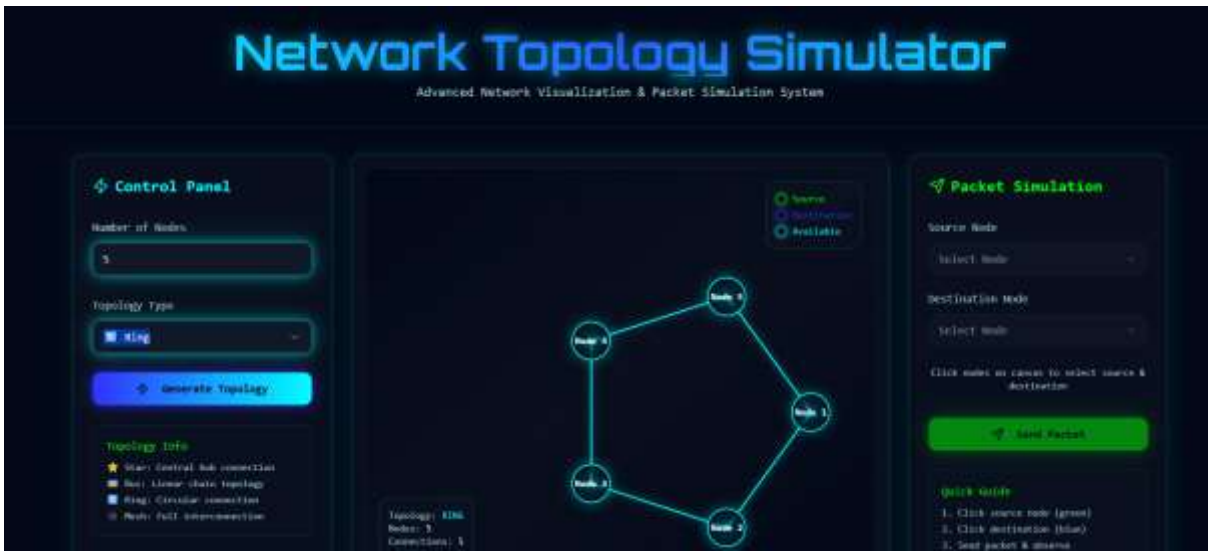


Layout: The screen is cleanly divided into three functional panels:

- **Left: Control Panel** (Configuration)
- **Centre: Topology Canvas** (Visualization)
- **Right: Packet Simulation** (Interaction)

- **Control Panel Functionality**

  The control panel demonstrates core configuration features. Users can set the **Number of Nodes** for the network and select the desired **Topology Type** (Star, Ring, Bus, or Mesh). An integrated **Topology Info** guide is visible, supporting the project's educational goal by providing quick characteristic definitions for each topology.



- **Topology Canvas & Visualization**

The central canvas validates the project's visualization capabilities. It clearly displays the generated structures, such as the centralized **Star Topology** (with a HUB) and the **Ring Topology** (as a closed loop). The interactive state is confirmed by the **color-coded legend** (Source, Destination, Available) which guides users through node selection.

- **Packet Simulation Panel**

This panel confirms the simulation workflow. It provides dedicated fields for **Source Node** and **Destination Node** selection. The prominent **Send Packet** button initiates the simulation process, triggering the network's internal path calculation. A **Quick Guide** provides clear, step-by-step instructions for the user interaction sequence.

7. TECHNICAL CHALLENGES & SOLUTIONS

7.1 Challenge: Animation Completion

Problem: Packet animation stopping before reaching destination *Solution*: Implemented dual-condition check:

- Monitor current node index in path
- Calculate physical distance to final destination
- Continue animation until both conditions satisfied

7.2 Challenge: Ring Topology Direction

Problem: Ambiguous routing in ring topology *Solution*: Enforced **unidirectional (clockwise)** routing rule matching real-world token ring behavior

7.3 Challenge: Path Highlighting

Problem: Highlighting correct connections during multi-hop transmission *Solution*: Path calculation integrated with connection rendering to identify and highlight active segments

8. FUTURE ENHANCEMENTS

8.1 Planned Functionalities

- Multi-packet concurrent simulation
- Custom user-defined topologies
- Collision detection and error simulation
- Statistics dashboard (hops, delay, throughput)
- Topology export (PNG/SVG)
- Integration with backend for saving user sessions

8.2 Performance Improvements

- Migrate SVG to **Canvas API** for large-scale networks
- Use **Web Workers** for heavy computation
- Apply **React.memo** to prevent unnecessary re-renders

9. CONCLUSION

The **Network Topology Simulator** marks a significant step toward bridging the gap between **theoretical networking concepts** and their **visual, practical understanding**. Through this project, we have successfully demonstrated the design, functionality, and behavior of four core network topologies — **Star, Bus, Ring, and Mesh** — in a fully interactive and visually engaging web-based environment.

The system not only focuses on the *structural visualization* of networks but also emphasizes the *logical and procedural aspects* of packet transmission, such as **path calculation, routing rules, and real-time animation**. This multi-dimensional approach transforms an otherwise abstract topic into an experiential learning process.

From a **technical perspective**, the simulator showcases the effective use of **React** and **TypeScript** for building dynamic, state-driven web applications. Leveraging modern tools such as **Vite** for optimized builds, **Tailwind CSS** for responsive design, and **SVG-based graphics** for precision rendering, the project demonstrates clean architecture and modular coding principles suitable for scalability and future development.

Moreover, the simulation engine provides an excellent foundation for exploring **network performance parameters**, **routing efficiency**, and **topological reliability**. The system's logging mechanism allows users to analyze transmission patterns, node interactions, and topology-specific advantages — fostering a deeper understanding of **network architecture design** and **data communication principles**.

10. REFERENCES & RESOURCES

10.1 Technical Documentation

- React Documentation: https://react.dev/
- TypeScript Handbook: https://www.typescriptlang.org/docs/
- Tailwind CSS: https://tailwindcss.com/docs
- Radix UI: https://www.radix-ui.com/
- Vite Documentation: https://vitejs.dev/guide/
- Node.js Documentation: https://nodejs.org/en/docs
- Express.js Guide: https://expressjs.com/en/guide/
- MongoDB Documentation: https://www.mongodb.com/docs/
- Firebase Documentation: https://firebase.google.com/docs

10.2 Network Topology Concepts

- *Computer Networks* — Andrew S. Tanenbaum & David J. Wetherall
- Network Topology Design Principles (IEEE & Cisco Whitepapers)
- Token Ring Protocol (IEEE 802.5)
- *Data and Computer Communications* — William Stallings
- OSI and TCP/IP Model References — RFC 1122, RFC 791
- Network Simulation Tools (e.g., Cisco Packet Tracer, GNS3)

10.3 Design & Development Resources

- Figma: https://www.figma.com/
- GitHub: https://github.com/
- Postman API Platform: https://www.postman.com/
- MDN Web Docs: https://developer.mozilla.org/

APPENDIX

A. Project Statistics

- Total Components: 8
- Lines of Code: ≈1,200+
- Supported Topologies: 4
- Maximum Nodes: 10
- Animation Frame Rate: 60 FPS
- Log Retention: 50 entries

B. Color Scheme (Cyberpunk Theme)

- Neon Green: hsl(var(--neon-green))
- Neon Blue: hsl(var(--neon-blue))
- Neon Cyan: hsl(var(--neon-cyan))
- Neon Pink: hsl(var(--neon-pink))